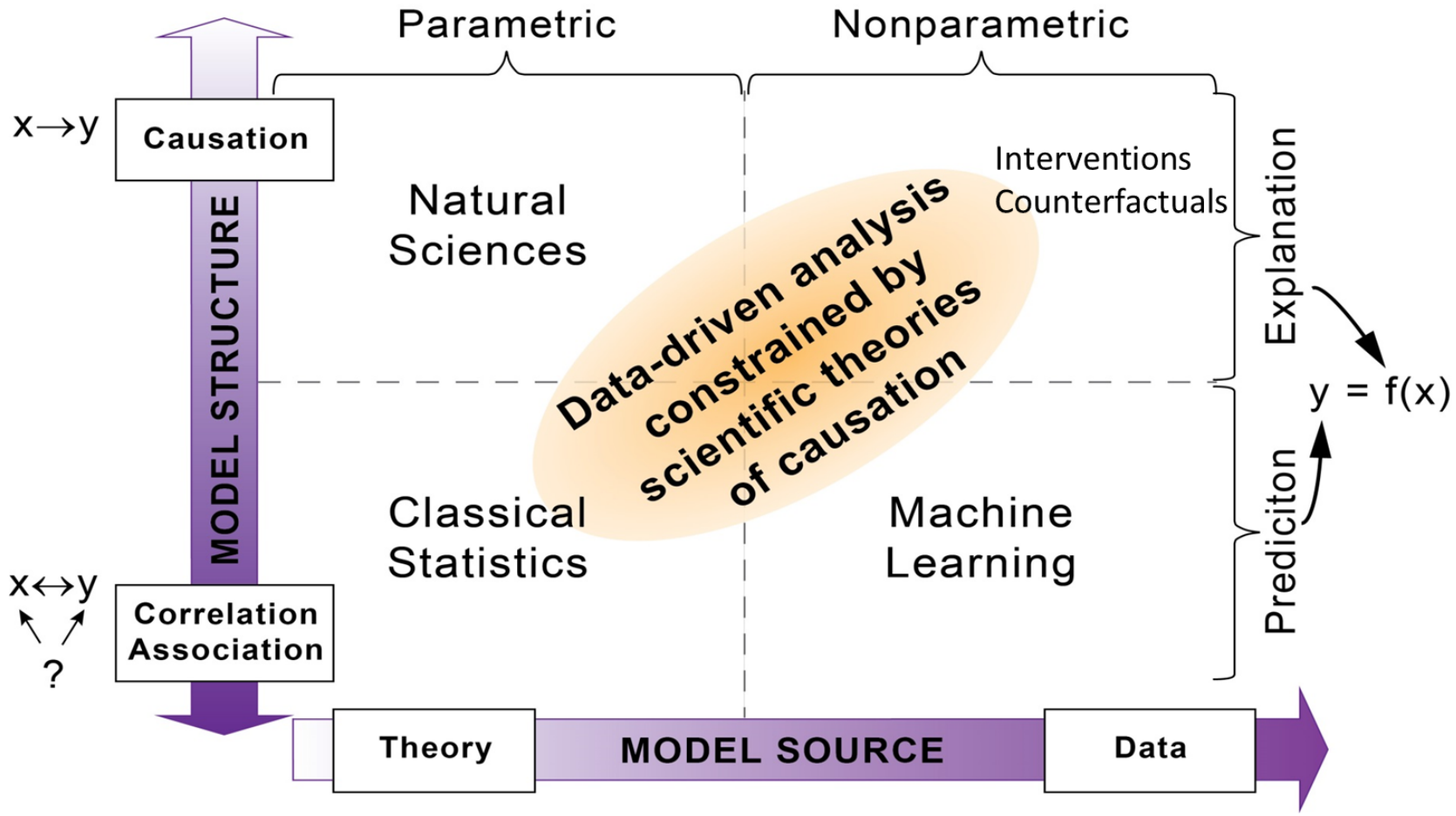
Introduction: What is Machine Learning?

Lecture 1

Tuesday March 3, 2026

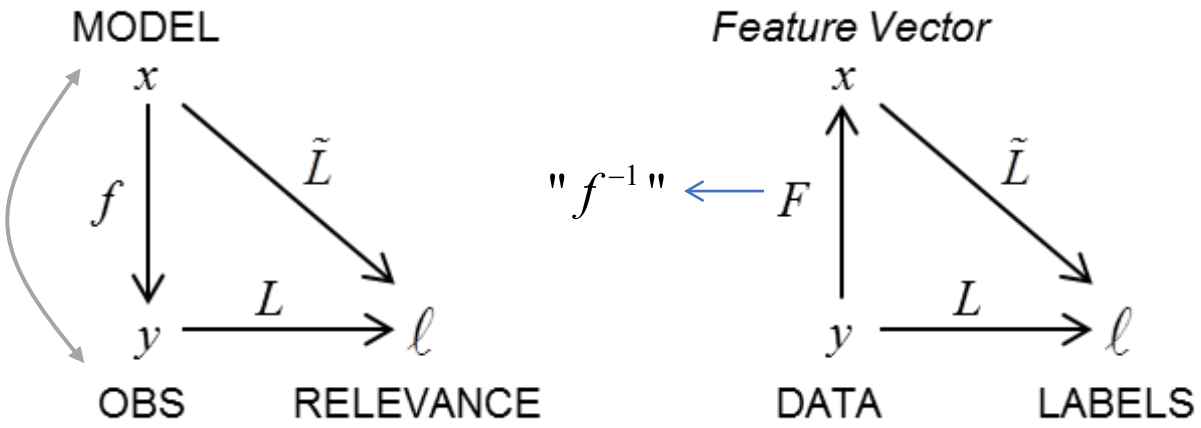
What is Machine Learning?



Connecting Science and Machine Learning

SCIENCE **MACHINE LEARNING**

Observations guide model building that guides experimental design to acquire useful observations.



Interpretability ***Predictive accuracy***

Physics Models and Machine Learning

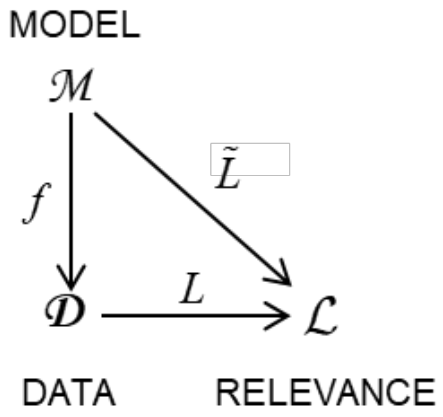
What are key properties of Physics Models?

- **Generative structure:** Predict effects of interventions (causal).
- **Compositionality:** Build complex behavior from local laws.
- **Cost:** Accurate simulation can be computationally prohibitive (multi-scale, stiff ODE/PDE, high-D configuration space).

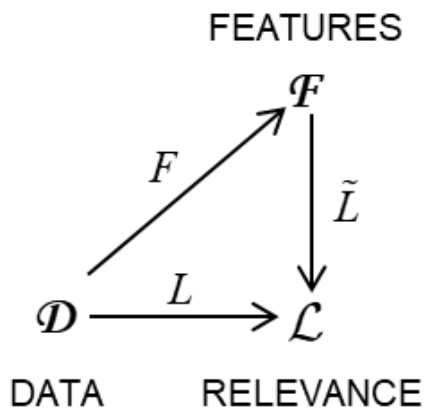
How can ML help Physics Models?

- **Model reduction/surrogates:** Drop irrelevant Degrees of Freedom (*DoF*) → Computational speed, validate for Out-Of-Distribution (OOD)/extrapolation.
- **Interpretable representations:** Preserve dominant mechanisms, constraints and symmetries → narrows model validity
- **Experimental flexibility:** Infer the Initial and Boundary Conditions (IC/BC) from data → Better match to the experiment.

Physics Model



Machine Learning



Lossy Compression is NOT Neutral

Lossy compression always requires:

- A distortion measure.
- A geometry.
- A notion of relevance.

Example: Minimize MSE,

$$R(X, \hat{X}) = \mathrm{E}\left\| X - \hat{X} \right\|^2$$

Hidden assumptions:

- Euclidean metric is meaningful.
- All coordinates are equally important.
- Squared error is the right notion of risk.

But in *scientific* data:

- Units differ.
- Calibration varies.
- Some variables matter more than others.
- Some invariants must be preserved.

Compression answers: What do we care about preserving? It is task-relative!

What is Machine Learning?

Definition 1: Machine learning replaces probability calculus with geometric operations on a vector space that exploits the efficiency of numerical linear algebra.

Definition 2: Machine learning replaces exact manipulation of probability measures with optimization in finite-dimensional function spaces, leveraging the computational efficiency of numerical linear algebra at the cost of strong geometric assumptions.

Probability Theory	**Machine Learning**
• Operates on measures	• Operate on vectors
• Uses integration	• Uses matrix multiplication
• Infinite-dimensional function space	• Finite-parameter spaces
• Calibrated uncertainty	• Performance on benckmarks
• Emphasize coherence	• Emphasize scalability
• Exact but intractable	• Approximate but computable

Key questions:

- What properties of probability calculus survive the linear algebra translation?
- What invariants are destroyed?
- When does geometric reasoning approximate measure-theoretic reasoning?
- When does it fail catastrophically?

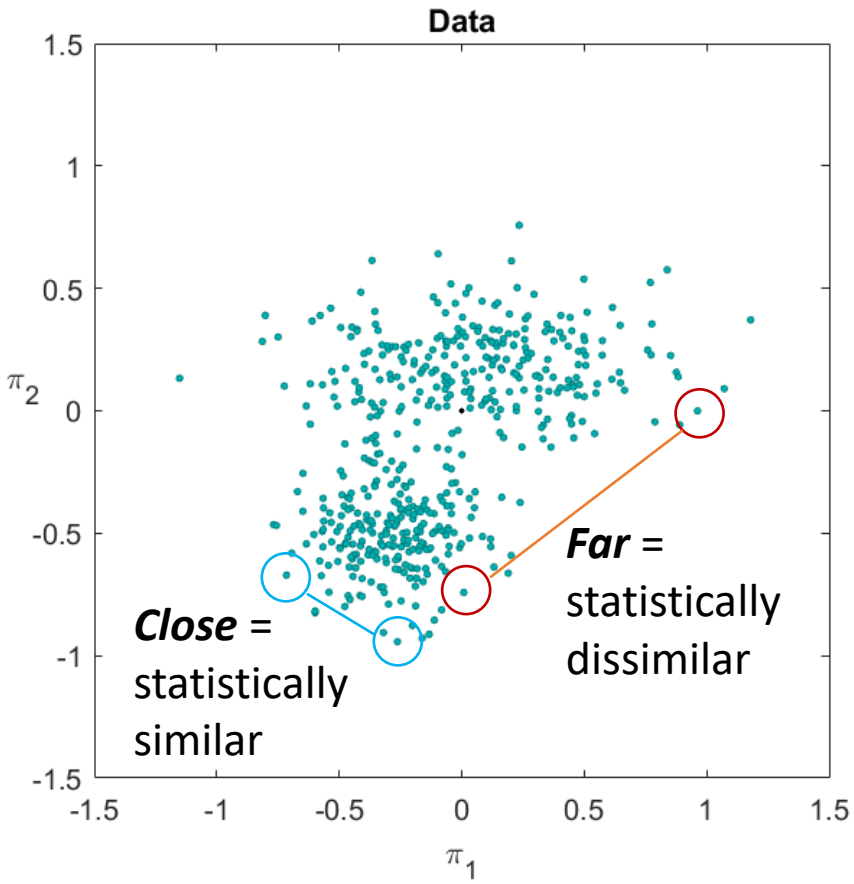
What is Machine Learning?

Create/Learn an Embedding or Feature Vector space that encodes information about *statistical similarity as distances*.

$$-\log p_{\text{similarity}}\left(y, y'\right) \approx \frac{1}{2}\left\|\mathbf{y}-\mathbf{y}'\right\|_2^2$$

How much DATA is enough?

If distant data points are mapped nearby in the model space, then there has to be significant inferential evidence (more data), and likewise if local data points are mapped to large separations by the model.

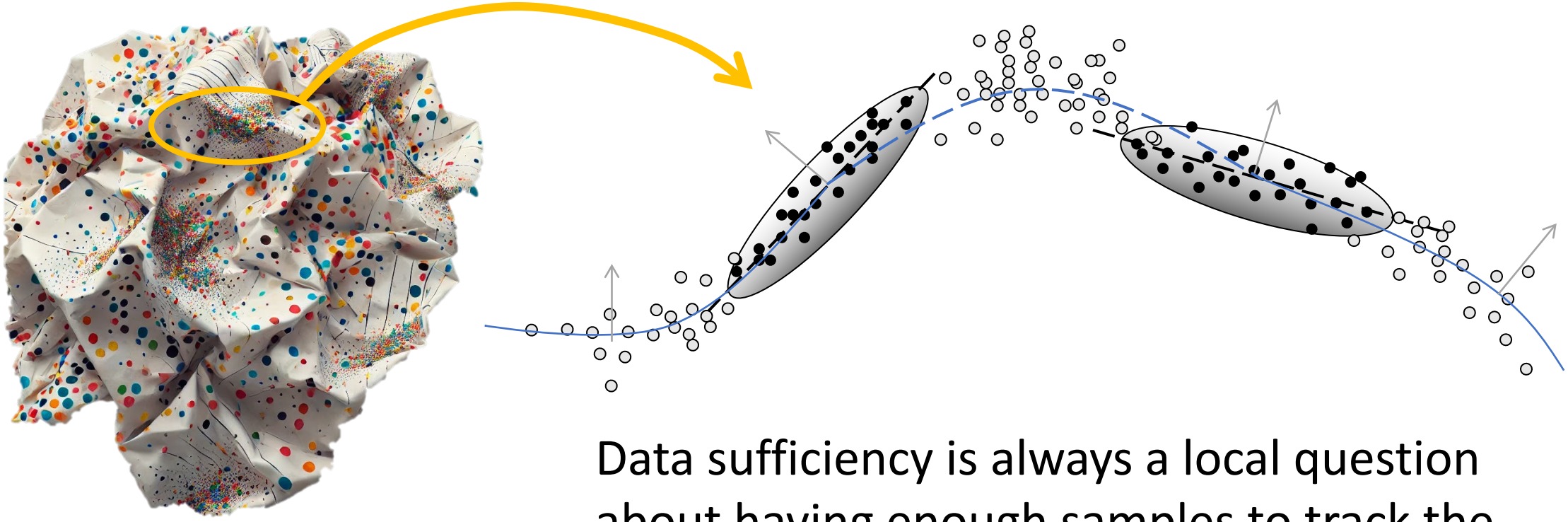


Embedding: Crumpled paper analogy

Machine Learning “Leap of Faith”: Non-vector data → Vector data

Do you have enough data to use ML?

"Crumpling" is the vector space embedding



Data sufficiency is always a local question about having enough samples to track the local curvature induced by the embedding

The Unifying Question

What is machine learning doing geometrically?

Generally, ML methods:

1. Place data into a space.
2. Impose a geometry on that space.
3. Use that geometry to solve a task.
4. The task is encoded as optimizing relative to a loss.

So the real questions are:

- Who chooses the geometry?
- What does the geometry preserve?
- What does the loss value?
- What assumptions are embedded in both?

The Supervised Learning Problem

To learn, we must make choices …

What is the Task we are trying to do? What is the mathematical representation of the Task? What data is relevant?

Consider a mapping between two domains:

$$X \xrightarrow{f} Y \qquad f : X \to Y$$

$$f : x \mapsto y, \text{ where } x \in X, \; y \in Y.$$

Represent the Task as a *Risk or Loss function* that defines the impact of being wrong: $R\left(\hat{f}(x), y\right)$ *Alignment problem*

The mapping f is unknown, however N associated pairs between the spaces X and Y are available.

Training data: $\mathcal{D} = \left\{(x_1, y_1), \ldots, (x_N, y_N)\right\}$ *Data sufficiency, Sampling complexity*

Select a parametric representation for the unknown function, f: $f_\theta(x)$ *Expressivity*

"Learn" by minimizing the Loss function, R, over the training data: $\min_\theta \mathrm{E}_{\mathcal{D}}\left[R\left(f_\theta(x), y\right)\right]$ *Fitting algorithm*

Since we optimized w.r.t the Loss function, we need to test with hold-out data: $\mathrm{E}_{\sim\mathcal{D}}\left[R\left(f_{\theta^*}(x), y\right)\right]$ *Post hoc uncertainty estimate*

Classical ML: Geometry First, Partition Second

Framework: $$f(x) = g(E(x)) = g \circ E(x)$$

- E: fixed embedding.
- g: learned partition or function.

Examples:
- PCA + regression.
- Kernel methods.
- Fixed Fourier/wavelet features.
- Local polynomial regression.

Properties:
- Geometry is explicit.
- Partition complexity is measurable.
- Attribution is clear.
- Bias from embedding can be separated from variance in partition.

Philosophically: We choose the coordinate system. Then we learn inside it.

Geometric Partitioning Schemes

A. Cluster-Based (Local Patches)

Assumption:
- Data concentrates in localized regions.

Pros:
- Clear attribution.
- Local uncertainty.
- Honest ignorance in sparse regions.

Cons:
- Complexity grows with volume.
- Weak extrapolation.

B. Low-Dimensional Subspace (Global)

Assumption:
- Data lies near a single linear manifold.

Pros:
- Parameter-efficient.
- Global compression.

Cons:
- Fragile to curvature.
- Diffuse attribution.
- Strong global prior.

C. Hyperplane Partitions

Assumption:
- Data separable by half-spaces.

Pros:
- Compact binary encoding.
- Decision-focused.

Cons:
- Global combinatorial complexity.
- Brittle boundaries.

Local Models	**Global Models**
Scale with data density	Share parameters globally
Honest uncertainty	Smooth extrapolation
Harder compression	Efficient compression

Question: Is failure to extrapolate a bug or a feature? In science, often a feature.

Modern Deep Learning: Learn the Geometry

<u>Deep networks remove the clean separation</u>

$$f(x) = g_\phi\left(E_\theta(x)\right) = g_\phi \circ E_\theta(x)$$

Both E and g are learned jointly adjusting their respective parameters, ϕ and θ, to minimize the loss function, R.

$$\min_\theta \mathrm{E}_{(x,y)\sim\mathcal{D}}\left[R\left(f_\theta(x), y\right)\right]$$

Only the loss function, L, is fixed.
This is a conceptual shift:
We no longer choose the geometry.
We let the loss shape it.

<u>Consequences</u>

- Geometry is task-shaped.
- Distances are not intrinsic.
- Embedding reflects loss, not ontology.
- Attribution becomes entangled.

New option: *Transfer Learning*
Assume that representations, E_θ, learned for one task can apply to a different task.

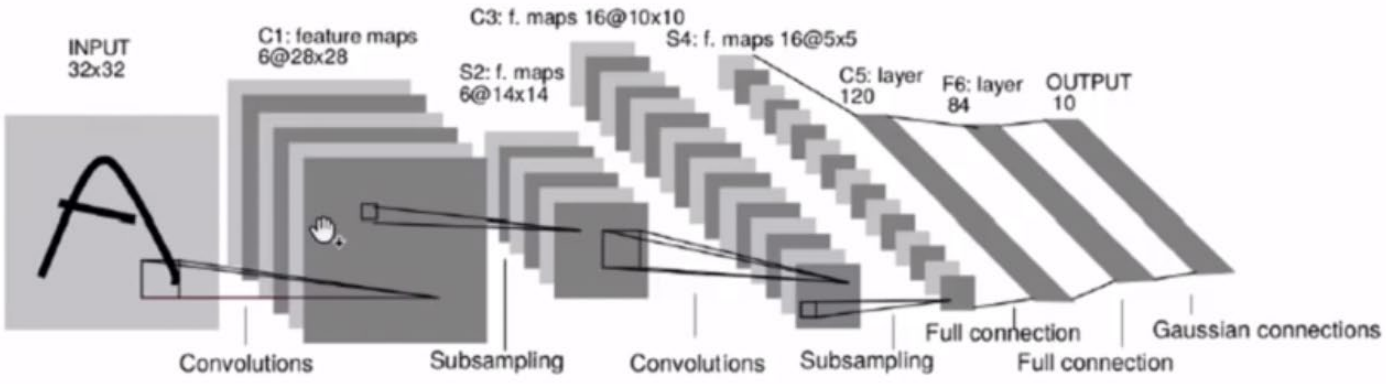
Surrogate objective function, R → Train to optimize R → Reuse transformation, E, of data to features for data not originally used to co-train g and E but learn a new h:

$$y = h_\phi \circ E_\theta(x)$$

Convolutional Neural Networks

Key properties of CNNs

Images
Patches
Pixels



- ☺ Convolutional **(Translation invariance)**
- ☺ Scale Separation **(Compositionality)**
- ☺ Filters localized in space **(Deformation Stability)**
- ☺ $\mathcal{O}(1)$ parameters per filter (independent of input image size n)
- ☺ $\mathcal{O}(n)$ complexity per layer (filtering done in the spatial domain)
- ☺ $\mathcal{O}(\log n)$ layers in classification tasks